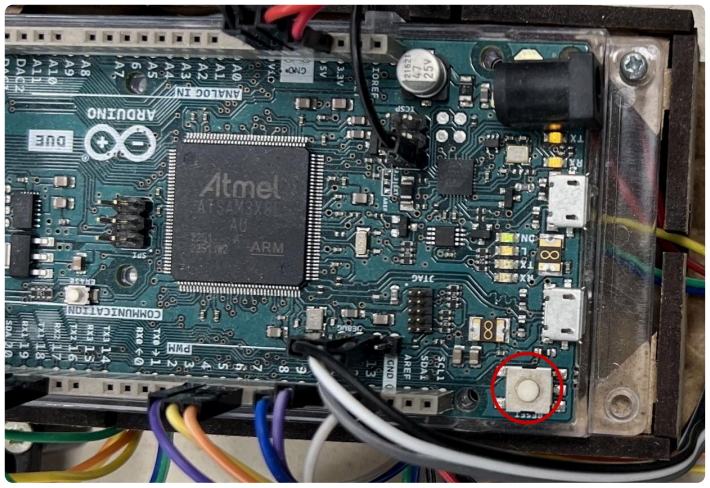
Intro

Before you can start coding you must first create a connection to the micro controller.

1. Start VS Code
    1. Open the `Terminal` > `Run Task` and type `OpenOCD`
    2. Add a breakpoint at line `58` in main.c
       within function `loop()`, line with `init()`
    3. Start debugging (**local**)
2. Open a new (external) terminal
    1. Type: `minicom -D /dev/ttyACM0` **OR** `minicom -D /dev/ttyACM1`
       One of them should work.
3. Press the reset button on the Arduino board



The output in the terminal window should be gibberish (random symbols).

4. Press `Ctrl + A + O` to open the Minicom options

```
        +-----------------+----------+----------+----------+----------+----------+
        | c        Clock | l    LED | m Motion | s  Sonic | 1-9 Goal | d  Debug |
+-------+-----------------+----------+----------+----------+----------+----------+
| St+-------------------------------------------------------------------------+ff |
+---| A - Serieller Anschluss      : /dev/ttyACM1                               |---+
| Va| B - Pfad zur Lockdatei       : /var/lock                                  |   |
+---| C - Programm zur Rufannahme  :                                            |---+
| Un| D - Programm zum Wählen      :                                            |   |
+---| E - Bps/Par/Bits             : 115200 8N1                                 |---+
    | F - Hardware Flow Control    : Nein                                       |
    | G - Software Flow Control    : Nein                                       |
    | H -     RS485 Enable      : No                                           |
    | I -   RS485 Rts On Send   : No                                           |
    | J -  RS485 Rts After Send : No                                           |
    | K -  RS485 Rx During Tx   : No                                           |
    | L -  RS485 Terminate Bus  : No                                           |
    | M - RS485 Delay Rts Before: 0                                            |
    | N - RS485 Delay Rts After : 0                                            |
    |                                                                          |
    |    Welchen Parameter möchten Sie ändern?                                 |
    +--------------------------------------------------------------------------+


CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.8 | VT102 | Offline | ttyACM1
```

> **Tip**
> Press *enter* to confirm and exit the menus

5. Change the baud rate to `115200`
    1. Press E
    2. Select `115200` from the sub-menu

2. Set `Hardware Flow Control` to **off**
    1. Press F
    3. Select `Save as dfl`
    4. Select the leave option to quit
6. Press the reset button again
7. Now you should see a table

```
        +----------------+----------+----------+----------+----------+----------+
        | c        Clock | l    LED | m Motion | s  Sonic | 1-9 Goal | d  Debug |
+-------+----------------+----------+----------+----------+----------+----------+
| State | Loop frequency | Blinking |      Off |      Off |        5 |      Off |
+-------+----------------+----------+----------+----------+----------+----------+
| Value |             44 |        1 |     0.00 |        0 |      500 |          |
+-------+----------------+----------+----------+----------+----------+----------+
| Unit  |             Hz |   On/Off |        % |       mm |       mm |          |
+-------+----------------+----------+----------+----------+----------+----------+


  CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.8 | VT102 | Offline | ttyACM1
```

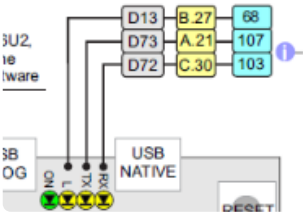
8. Press the keys displayed on the left in the columns of the first row to switch between options
9. To quit Minicom type `Ctrl + A + Q`

Lab 1

> In this lab you're supposed to implement some functionality for `led.c`.

Task 1

The first step is to initialize the LED which is located on the Arduino DUE board.



This is an excerpt from the board pinout for the **Arduino DUE**. As you can see, if you take a look at the full image, the LED is located next to the round reset button, one which you already used during the setup (**Lab #0**). There's other LEDs which have specific meaning tied to them, e.g. TX can be inferred to be used for when information has been transmitted.
The one we will be using is simply labelled with an L.

You can ignore the digital and physical pin designations, we will only focus on the port pin (which is the yellow one). Through the capital letter we can tell that the pin is controlled by the **PIO controller B** and the number after it tell us that the LED uses the controller's **27th I/O line** or perhaps it helps to think in terms of bits, then one can also say that the LED corresponds to the **27th** bit in the registers that are part of the **PIO Controller User Interface**.

Now, when you look through the datasheet (specifically ch. **9.3**) you will find our I/O line 27 is multiplexed with two functions. However, we only want to use the LED, therefore the **PIO status register** (`PIO_PSR`) should return 0 for bit 27.

- **P0-P31: PIO Status**

0: PIO is inactive on the corresponding I/O line (peripheral is active).

1: PIO is active on the corresponding I/O line (peripheral is inactive).

How do we set the status? By setting the bit (*here:* 27) in the **PIO enable register** (`PIOB_PER`) or the **PIO disable register** (`PIOB_PDR`).
What's the difference? The enable register enables the PIO controller and thus allows it to control the I/O line. Conversely the disable register enables the peripheral and gives it control over the I/O line (this is what those other functions refer to).

Code

Initialisation

1. First we have to enable the pin (give control to the PIO controller). We use an OR-operator to avoid overwriting any bits that were previously set (technically we don't have to in this instance but it's still good practice).
   `PIOB_PER |= DUE_LED_BUILTIN;`
   `DUE_LED_BUILTIN` is defined as `0x08000000` -- it's identical to left-shifting 1 by 27 bits.
2. Then we select which data direction we want the pin to have (input or output).
   If we want *output* we simply set the **output data register** with bit 27.
   If we want an *input*, however, we must set the **output disable register** with bit 27.
   In our case we have an LED, which means output, so `OER`.
   `PIOB_OER |= DUE_LED_BUILTIN;`
3. Next we want to enable the power management controller (ch. **28**) for **PIO controller B**. You don't necessarily have to for pins set to output but it's best practice to do it anyways.

"The Power Management Controller (PMC) optimizes power consumption by controlling all system and user peripheral clocks. The PMC enables/disables the clock inputs to many of the peripherals and the Cortex-M3 Processor."
-- datasheet chapter 28
So, the PMC is what makes the PIO controllers, peripherals etc usable.
Get LED status

4. Now we want to get the current status of the LED. For that we need to make use of the **output data status register** since it contains the data that is being transmitted to the LED.
   - **P0-P31: Output Data Status**

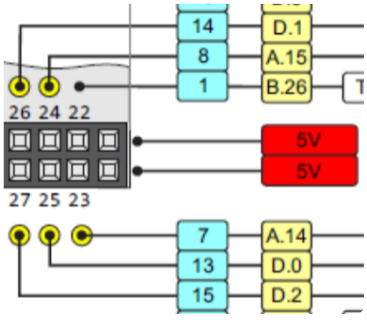
   0: The data to be driven on the I/O line is 0.

   1: The data to be driven on the I/O line is 1.

   So, we want to know if the 27th bit in the output data status register contains a 0 or a 1, with 0 meaning off and 1 on. Since we want to know **ONLY** the status of our LED we have to mask the bits, e.g. by using our DUE_LED_BUILTIN constant. To do this we utilize the bit-wise AND-Operation via the ampersand (&).
   If ODSR yields 0x0 then 0x0 & 0x00001... will return 0x0.
   If it's something other than 0x0 then it will return 0x00001...(`0x08000000`).
   Therefore, all you need to return is `PIOB_ODSR & DUE_LED_BUILTIN` .
   Set LED status
5. This should be simple enough. We have...
   1. a function parameter, so our new value that we want to pass to the LED,
   2. the global variable `status.led` to which we pass the new value,
   3. and the registers `SODR` and `CODR` . They allow for setting and clearing of the **output data register**.
      `if(led_on) {`
      `PIOB_SODR |= DUE_LED_BUILTIN; } else {`
      `PIOB_CODR |= DUE_LED_BUILTIN; }`

Task 2

According to the exercise sheet the buttons 1 to 3 are wired to the port pins D0, D1 and D2 respectively. Here another excerpt of the board pinout for those who care.



Since we are doing the same thing as before, only with slightly different configuration we will skip explaining most of the code.

Code

Expand the init

1. Enable power management for PIO controller D
   `PMC_PCER0 |= PMC_PCER0_PID14;`
2. Enable PIO controller
   `PIOD_PER |= PIOD_PER_P0 | PIOD_PER_P1 | PIOD_PER_P2;`
3. Set data direction to input **by disabling output**
   `PIOD_ODR |= PIOD_ODR_P0 | PIOD_ODR_P1 | PIOD_ODR_P2;`

4. Enable pull up resistor. This makes it so the pin has a known voltage (is not floating), meaning the pin is not going to randomly read a 0 or 1 (due to noise) but is always set to 0 or 1. In this case we have a pull-up resistor, so it drives the line high (1).
   `PIOD_PUER |= PIOD_PUER_P0 | PIOD_PUER_P1 | PIOD_PUER_P2;
   Loop
5. We first do the polling variant. For that we use if-else branches to check which of the three buttons have been pressed. Since the button press drives the line to low (0), we need to negate the **pin data status register** in order to check if the button has been pressed.
   The logic for the button is thus inverted. When the PDSR reads 1 for the respective bit then that means the button was not pressed. To correctly implement the checks we simply negate the result of the (bit-wise) AND-operation.
   We also use constants for the buttons for better readability, the constants have the same values as `PIOD_PDSR_P0`, `PIOD_PDSR_P1` and `PIOD_PDSR_P2` (c.f. `sam.h`).
   `(!(PIOD_PDSR & DUE_BUTTON_1))`
   Duplicate this line 2 times for the other buttons. Then in the branches set the state of the LED, e.g. `state.led = LED_STATE_OFF;`.
   Expand the init Part 2
6. Let's say you want your program to change the LED's state only when a button has been pressed. That requires an interrupt handler. For that we use the **Nested Vectored Interrupt Controller (NVIC)**. We want to enable interrupt handling for the **PIO controller D**, for that we need to set the corresponding bit which is already conveniently defined as a constant in the `sam.h`.
   `NVIC->ISER[0] |= 1 << PIOD_ID;`
7. Then we only need to specify for which I/O lines on the PIOB we want to enable the **Input Change Interrupt**. Since it's P0-P2 that means bit 0, 1 and 2 have to be set which corresponds to 0...0111 and 7 in hexadecimal.
   `PIOD_IER = 0x7;`
   Interrupt handler
8. Now we have to implement our interrupt handler `PIOD_Handler()` (c.f. `exception.c`). This is rather simple as we only save the interrupt status register in a local, volatile and unsigned integer variable `status` and then move the code from **5.** out of the loop and into the handler function. Then replace `PIOD_PDSR` with `status` and remove the negation.

Leitfragen

1. Wofür stehen die einzelnen Registerabkürzungen und welchen Nutzen haben diese?
   Explained above, otherwise look through the datasheet.
2. Wieso muss Peripherie im PMC aktiviert werden?
   Because the clock is disabled by default for most components (this is more energy efficient) and so to use those components the clock has to be manually enabled.
3. Welchen Unterschied gibt es zwischen der Nutzung von Busy Waiting und Polling? Was sind die Vor- und Nachteile?
   Busy-waiting: The program continuously checks the component (the button) in a loop.
   Pro: Quick reaction since it does nothing else.
   Con: Can't do anything else and also uses up all CPU resources.
   Polling: The program checks the status of the peripheral component after a certain amount of time and then continues on with the rest of its instructions.
   Pro: Uses comparatively few resources and allows for other code to be executed as well.
   Con: Doesn't react instantly with the possibility to not react at all , e.g. when the button is pressed while the program is running a different loop then theoretically it could ignore / not register the button press.
   Interrupt: The program runs other code until it receives the signal (interrupt) upon which it starts running the code in the interrupt handler. After all that has been executed the program returns to where it was prior to the interrupt.
   Pro: Quick reaction and efficient use of CPU resources
   Con: Potentially more complex to set up



Lab 2

> In this lab you're supposed to implement I²C communication to drive the **HT16K33A** LED matrix via `matrix.c`. The matrix will later be used as a progress bar to visualize distance measurements from the ultrasonic sensor (Lab 3).

On the Arduino DUE, I²C is implemented by the **Two-Wire Interface (TWI)** peripheral. TWI and I²C refer to the same bus protocol. Unlike USART, I²C allows multiple slave devices on the same two wires (SDA and SCL).
The LED matrix is connected to **TWI1** on pins **PB12** (TWD1 / SDA) and **PB13** (TWCK1 / SCL). The matrix controller's slave address is **0x70** (configurable up to 0x77 via solder jumpers on the board).

Task 1

Initialize the I²C interface and the attached LED matrix in `matrix_init()`.
Before TWI1 can drive the bus, its clock must be enabled in the PMC, the pins must be handed over to the peripheral, and the TWI controller itself must be configured in master mode.

Code

Enable TWI1 clock

1. Enable the peripheral clock for TWI1. TWI1 has peripheral ID **23** (c.f. datasheet ch. **9**).
   `PMC_PCER0 = PMC_PCER0_PID23;`

Configure PIO pins (pre-handover)

2. Enable PIO control for the SDA and SCL pins, set them as outputs, and define their idle levels. This is done *before* the pins are assigned to the TWI peripheral — the datasheet recommends clearing SDA and setting SCL high as the default bus state.

```
PIOB_PER = PIOB_PER_P12 | PIOB_PER_P13;
PIOB_OER = PIOB_OER_P12 | PIOB_OER_P13;
PIOB_CODR = PIOB_CODR_P12;
PIOB_SODR = PIOB_SODR_P13;
```

Reset and configure TWI1

3. Disable the PDC (DMA) channels, then perform a software reset of the TWI controller.

```
TWI1_PTCR = TWI1_PTCR_RXTDIS | TWI1_PTCR_TXTDIS;
TWI1_CR = TWI1_CR_SWRST;
```

4. Set the clock waveform generator (`TWI1_CWGR`) for the desired bus speed. The reference solution uses fast mode:

```
TWI1_CWGR = TWI1_CWGR_CKDIV(3) | TWI1_CWGR_CHDIV(13) | TWI1_CWGR_CLDIV(13);
```

5. Configure the master mode register (`TWI1_MMR`) with the matrix slave address. `IADRSZ_NONE` means no internal address bytes are sent — the HT16K33A expects raw command bytes.

```
TWI1_MMR = TWI1_MMR_IADRSZ_NONE | TWI1_MMR_DADR(0x70);
```

6. Enable master mode and the transmit PDC channel.

```
TWI1_CR = TWI1_CR_MSEN;
TWI1_PTCR = TWI1_PTCR_TXTEN;
```

Hand pins to TWI1 peripheral

7. Select peripheral function A for both pins (`PIOB_ABSR` bit = 0) and disable PIO control via `PIOB_PDR`. After this, TWI1 drives SDA and SCL directly.

```
PIOB_ABSR &= ~(PIOB_ABSR_P12 | PIOB_ABSR_P13);
PIOB_PDR = PIOB_PDR_P12 | PIOB_PDR_P13;
```

Initialize the HT16K33A
8. Send the controller's init sequence. Each byte is a separate I²C write (see Task 2):
- `0x21` — turn on the internal oscillator
- `0xA0` — set row output pins
- `0xE0` — set dimming
- `0x81` — disable blinking, display on
9. Clear the display buffer and send it to the matrix.

Task 2

Implement the actual I²C transmission inside `matrix_write()`. You can use either the **Transmit Holding Register (THR)** with polling on the **Status Register (SR)**, or the **PDC/DMA controller** with **TPR** (Transmit Pointer Register) and **TCR** (Transmit Counter Register).

The reference solution uses the DMA approach: set the pointer to the buffer, set the byte count, then issue a STOP condition to complete the transfer.

Code

Check readiness

1. Before starting a new transfer, poll `TWI1_SR` for `TWI1_SR_TXCOMP` (Transmission Completed). This bit is set when the previous transfer has finished.
   `return (TWI1_SR & TWI1_SR_TXCOMP);`

Start DMA transfer
2. Write the buffer address and length into the PDC registers, then send a STOP condition via the control register.
`TWI1_TPR = (unsigned int)buf;`
`TWI1_TCR = n;`
`TWI1_CR = TWI1_CR_STOP;`

THR alternative
3. Instead of DMA, you can write each byte directly to `TWI1_THR` and poll `TWI1_SR_TXRDY` before each write. The CPU must then copy every byte manually, whereas DMA reads directly from RAM.

Task 3

Verify `matrix_write()` by displaying your lab computer number on the LED matrix inside `matrix_loop()`.

The HT16K33A drives an 8×8 LED matrix. The display buffer has 17 bytes: byte 0 is unused, and bytes 1, 3, 5, …, 15 each control one row (only the lower 8 bits of each row byte are used). To draw a digit or pattern, set the appropriate row bytes and call `matrix_write()`.

Code

Display a pattern when sonic is off

1. When `state.sonic == SONIC_STATE_OFF`, fill the buffer with a fixed pattern that encodes your lab PC number. Each row byte lights up specific LEDs in that row.

`buffer[1] = 0xFF;` — row 0 fully lit
`buffer[3] = 0xFF;` — row 1 fully lit
`buffer[5] = 0x06;` — row 2: segments for digit shape
… and so on for rows 3–7.

2. Always push the buffer to the matrix at the end of the loop:
`matrix_write(buffer, sizeof(buffer));`

> Experiment with different row values to find the combination that displays your PC number. Setting all elements to `0xFF` lights the entire matrix; partial values create specific digit shapes.

Task 4

Implement the distance display in `matrix_distance()`. When the ultrasonic sensor is on (`state.sonic == SONIC_STATE_ON`), show the remaining distance as a progress bar. When sonic is off, fall back to the lab PC number from Task 3.

The progress bar fills rows from bottom to top. Each row holds 8 LEDs, so a distance of e.g. 20 means 2 full rows plus 4 LEDs in the next row.

Code

Blink on negative distance

1. If the distance minus goal is ≤ 0, enable blinking on the matrix to signal the target has been reached or overshot.
`matrix_blinking(true);`

Map distance to buffer
2. Call `show_led_nr()` to convert the distance value into row bytes. The `decimal[]` lookup table maps a count (0–8) to the correct bit pattern for a partially filled row.
`show_led_nr(dist);`

3. Write the updated buffer to the matrix.

From `matrix.c` :

```
static bool matrix_distance(int dist)
{
    if (!matrix_ready())
        return false;

    static bool blink = false;
    // start blinking when value is negative
    if (!blink && dist <= 0)
        while (!matrix_blinking(blink = true))
            ;
    else if (blink && dist > 0)
        while (!matrix_blinking(blink = false))
            ;

    // invert distance if negative
    if (dist <= 0)
        dist = -dist;

    show_led_nr(dist);
    matrix_write(buffer, sizeof(buffer));
```

```
    return true;
}
```

`show_led_nr()` **— fill rows bottom-up**

4. Compute how many full rows and leftover LEDs the distance requires, then write the corresponding `decimal[]` entries into the buffer.

```
void show_led_nr(int count)
{
    int full_rows = count / 8;
    int leftover_leds = count % 8;
    int temp = 0;

    for (unsigned int i = 0; i < full_rows; i++)
    {
        if (i == 0)
        {
            buffer[i + 1] = decimal[8];
        }
        temp = ((i * 2) + 1);
        buffer[temp] = decimal[8];
    }

    int index = 0;
    if (full_rows == 0)
    {
        index = 1;
    }
    else
    {
        index = temp + 2;
    }

    buffer[index] = decimal[leftover_leds];
}
```

The `decimal[]` table at the top of `matrix.c` encodes how many LEDs are lit per row in base 2.

Leitfragen

1. Welche Adresse müssen Sie für die Ansteuerung der Matrix verwenden und wie wird diese konfiguriert?
   Die HT16K33A hat die I²C-Slave-Adresse **0x70** (bis 0x77 je nach Jumper). Sie wird im **Master Mode Register** (`TWI1_MMR`) über das Feld `DADR` gesetzt: `TWI1_MMR_DADR(0x70)` . Kein internes Adressbyte (`IADRSZ_NONE`), da der Controller direkt Kommandobytes erwartet.
2. Wie läuft die eigentliche Übertragung über I²C ab?
   Der TWI-Master sendet eine **START-Bedingung**, die 7-Bit-Slave-Adresse (0x70), dann die Nutzdaten (Kommandobytes oder Display-Buffer), und schließlich eine **STOP-Bedingung** (`TWI1_CR_STOP`). Der Statusregister-Bit `TXCOMP` signalisiert, dass die Übertragung abgeschlossen ist.
3. Welchen Unterschied gibt es bei der Verwendung des DMA-Controllers anstelle des THR Registers?
   Mit **THR** muss die CPU jedes Byte einzeln in das Holding Register schreiben und auf `TXRDY` warten. Mit **DMA (TPR/TCR)** zeigt der Pointer auf einen Speicherbereich und der Controller liest die Bytes automatisch — die CPU ist während der Übertragung frei für andere Aufgaben.
4. Erkennen Sie Unterschiede in der Loop_Frequenz?
   Ja. Jeder `matrix_write()` -Aufruf blockiert mindestens so lange, bis `TXCOMP` gesetzt ist. Häufiges Schreiben des gesamten 17-Byte-Buffers in jeder Schleifeniteration reduziert die Loop-Frequenz messbar (über `state.clock == CLOCK_STATE_LOOP_FREQUENCY` prüfbar).

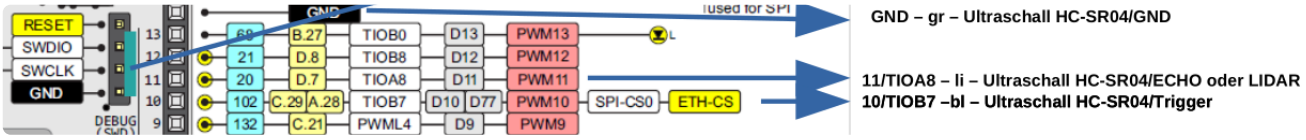
5. Wie ist die Ausgabe auf das Terminal realisiert?
   Die serielle Ausgabe läuft über **USART** in `serial.c`. Die Matrix-Kommunikation ist davon unabhängig und nutzt den separaten TWI1-Bus. Status- und Messwerte werden über das Menü (`menu.c`) und `state_loop()` auf dem Terminal angezeigt.



Lab 3

> In this lab you're supposed to implement distance measurement with the **HC-SR04** ultrasonic sensor via `sonic.c`. The sensor is triggered by a periodic pulse on **TIOB7 (PIOC29)** and the echo length is captured on **TIOA8 (PIOD7)** using Timer Counter 2 in wave and capture mode.

The HC-SR04 works by sending a 10 µs trigger pulse, then measuring how long the echo pin stays high. That time corresponds to the round-trip distance to an obstacle. Speed of sound is approximately **343 m/s** at 20 °C.



Task 1

Initialize the timer/counter blocks in `sonic_init()` so that **TIOB7** generates the trigger signal and **TIOA8** captures the echo pulse length.

Two separate TC2 channels are used: **channel 1** for the trigger output (wave mode) and **channel 2** for the echo input (capture mode).

Pinout and peripheral IDs

```
| Signal  | Pin             | TC channel    | Peripheral ID (PMC) |
|---------|-----------------|---------------|---------------------|
| Trigger | PIOC29 / TIOB7 | TC2 channel 1 | PID34 (`PMC_PCER1`) |
| Echo    | PIOD7 / TIOA8  | TC2 channel 2 | PID35 (`PMC_PCER1`) |
```

36.5.3 Interrupt Sources

The TC has an interrupt line connecte
the interrupt controller before configuri

Table 36-5. Peripheral IDs

Instance	ID
TC0	27
TC1	28
TC2	29

Code

Trigger init (`trigger_init`)

1. Disable the timer clock, enable PIO control of PIOC29, set it as output, and clear the output data register (drive low).
   `TC2_CCR1 = TC2_CCR1_CLKDIS;`
   `PIOC_PER = PIOC_PER_P29;`

`PIOC_OER = PIOC_OER_P29;`
`PIOC_CODR = PIOC_CODR_P29;`

2. Select peripheral B for the pin (TIOB function) and enable the TC2 clock.
`PIOC_ABSR |= PIOC_ABSR_P29;`
`PMC_PCER1 |= PMC_PCER1_PID34;`
3. Configure channel 1 in **wave mode** (ch. **36**): count up to RC, output TIOB. On RB compare the output clears (start of pulse), on RC compare it sets (end of period). This produces a periodic trigger pulse.
`TC2_CMR1 = TC2_CMR1_WAVE | TC2_CMR1_WAVSEL_UP_RC | TC2_CMR1_EEVT_XC0 | TC2_CMR1_BCPB_CLEAR | TC2_CMR1_BCPC_SET | TC2_CMR1_TCCLKS_TIMER_CLOCK1;`
4. Set the period and pulse width. At MCK/2 = 42 MHz: RC = 4_200_000 → 100 ms period (10 Hz), RB = 42_000 → 1 ms pulse (well above the 10 µs minimum).
`TC2_RC1 = TRIGGER_RC;`
`TC2_RB1 = TRIGGER_RB;`
5. Hand the pin to the peripheral and start the timer.
`PIOC_PDR = PIOC_PDR_P29;`
`TC2_CCR1 = TC2_CCR1_CLKEN | TC2_CCR1_SWTRG;`

Echo init (`echo_init`)

6. Disable timer clock, enable PIOD7 as input, select peripheral B.
`TC2_CCR2 = TC2_CCR2_CLKDIS;`
`PIOD_PER = PIOD_PER_P7;`
`PIOD_ODR = PIOD_ODR_P7;`
`PIOD_ABSR |= PIOD_ABSR_P7;`
`PMC_PCER1 |= PMC_PCER1_PID35;`
7. Configure channel 2 in **capture mode**: load RA on rising edge, RB on falling edge. `LDBDIS` disables the clock after the falling edge is captured — the echo loop must restart it.
`TC2_CMR2 = TC2_CMR2_TCCLKS_TIMER_CLOCK1 | TC2_CMR2_LDRA_RISING | TC2_CMR2_LDRB_FALLING | TC2_CMR2_LDBDIS;`
8. Hand pin to peripheral and enable the clock.
`PIOD_PDR = PIOD_PDR_P7;`
`TC2_CCR2 = TC2_CCR2_CLKEN | TC2_CCR2_SWTRG;`

> The reason we set PER, configure output, and clear the pin in `trigger_init` is that this init runs before `trigger_loop`, which calls `trigger_enable` / `trigger_disable`. We must configure the default PIO state before handing the pin to the timer peripheral.

Task 2

Read the echo pulse length inside `sonic_loop()` using registers **RA**, **RB**, and the **Status Register (SR)**. Use polling (check `LDRBS` each loop iteration). A new capture can be started via **CCR** (`CLKEN | SWTRG`).

Code

Enable/disable trigger

1. When sonic is off, return PIO control of the trigger pin (stop pulses). When on, hand it back to the timer.
`trigger_disable()` → `PIOC_PER = PIOC_PER_P29;`
`trigger_enable()` → `PIOC_PDR = PIOC_PDR_P29;`

Poll for completed capture

2. Read `TC2_SR2` once per loop. The `LDRBS` bit indicates that a falling edge was captured into RB.
`unsigned int SR = TC2_SR2;`
`if (SR & TC2_SR2_LDRBS) { ... }`

Calculate distance

3. Read RA (rising edge timestamp) and RB (falling edge timestamp). The difference is the echo pulse width in timer ticks.

`unsigned int diff = TC2_RB2 - TC2_RA2;`

4. Convert ticks to time, then to distance. The timer runs at MCK/2; the echo time is a round trip, so divide by 2.

`float time = diff / (double)MCK;`

`float distance = (float)SONIC_SPEED * time;`

`value.sonic.distance = distance * 1000.0f;` — store in millimetres

`value.sonic.new = true;`

Restart capture

5. After reading, restart the echo timer for the next pulse (required because `LDBDIS` stops the clock).

`TC2_CCR2 = TC2_CCR2_CLKEN | TC2_CCR2_SWTRG;`

Task 3

Implement the same measurement using **interrupts** instead of polling. Write a `TC8_Handler()` (or `TC2_Handler()` depending on which TC block your echo channel uses) and enable it via the NVIC **Interrupt Set-Enable Register (ISER)**.

Code

Enable the interrupt

1. Enable the TC interrupt in the NVIC. For TC2 channel 2, use the corresponding TC IRQ number from ch. **9**.
   `NVIC->ISER[0] |= 1 << TC2_ID;`
2. Enable the load-B interrupt in the TC interrupt enable register.
   `TC2_IER2 = TC2_IER2_LDRBS;`

Interrupt handler

3. In the handler, read RA and RB, compute the distance, set `value.sonic.new = true`, then restart the capture.

```
void TC2_Handler(void) {
unsigned int RA = TC2_RA2;
unsigned int RB = TC2_RB2;
unsigned int diff = RB - RA;
float time = diff / (double)MCK;
value.sonic.distance = (float)SONIC_SPEED * time * 1000.0f;
value.sonic.new = true;
TC2_CCR2 = TC2_CCR2_CLKEN | TC2_CCR2_SWTRG;
}
```

4. Remove the polling logic from `echo_loop()` — the handler takes over measurement. `echo_loop()` can then be empty (or only handle the sonic-off case).

> The vector table in `exception.c` already reserves a slot for `TC2_Handler` and `TC8_Handler`. Override the weak alias by providing your own implementation.

```
void TC2_Handler(void) __attribute__((weak, alias("Dummy_Handler")));
// ...
void TC8_Handler(void) __attribute__((weak, alias("Dummy_Handler")));
```

Polling vs. Interrupt

- **Polling** (reference solution): simple, no NVIC setup, but checks `SR` every loop iteration even when no echo arrived.
- **Interrupt**: CPU only reacts when a falling edge is captured — more efficient, but requires correct NVIC and IER configuration.

Task 4

Display the measured distance on the LED matrix when a new measurement is available and `state.sonic == SONIC_STATE_ON`.

This is handled in `matrix_loop()` (Lab 2). When sonic is on and `value.sonic.new` is set, the matrix shows the remaining distance (distance minus goal) as a progress bar.

Code

1. Check that sonic is enabled and a new measurement flag is set.
   `if (value.sonic.new) { ... }`
2. Clear the matrix buffer, then call `matrix_distance()` with the difference between measured distance and goal.
   `matrix_distance((int)value.sonic.distance - (int)value.goal);`
3. Reset the new-measurement flag.
   `value.sonic.new = false;`

> Toggle sonic on via the serial menu. With `state.sonic == SONIC_STATE_ON`, the matrix fills row by row as you move closer to the goal distance.

Leitfragen

1. Welche Distanzen können gemessen werden?
   Der HC-SR04 misst typischerweise **2 cm bis ca. 4 m**. Unter 2 cm ist das Echo zu kurz für eine zuverlässige Erfassung; über 4 m wird das Signal zu schwach. Die maximale Echozeit bei 4 m beträgt ca. 23 ms (Hin- und Rückweg).
2. Welche Zeiten benötigen Abstandsmessungen?
   Ein kompletter Messzyklus braucht mindestens die **Triggerperiode** (100 ms bei 10 Hz im Referenzcode) plus die **Echozeit** (ca. 116 µs pro cm Distanz). Bei 10 Hz Triggerfrequenz hat jede Messung genug Zeit, bevor der nächste Puls kommt.
3. Welches Verfahren (Busy-Waiting, Polling oder Interrupt) werden Sie für die Projektlösung verwenden?
   Für die Projektlösung eignet sich **Polling** oder **Interrupts** — beides ist besser als Busy-Waiting, da die Hauptschleife weiterlaufen kann. Polling ist einfacher zu debuggen; Interrupts sind effizienter, weil die CPU nicht in jeder Iteration `TC2_SR2` prüfen muss. Die Referenzlösung nutzt Polling in `echo_loop()`.
4. Warum wird der Trigger mit 10 Hz erzeugt und nicht schneller?
   Der HC-SR04 braucht nach jedem Puls genug Zeit für das Echo-Signal. Bei hohen Frequenzen können Echos von der vorherigen Messung noch andauern, wenn das Hindernis weit entfernt ist. 10 Hz (100 ms Periode) stellt sicher, dass auch entfernte Objekte korrekt gemessen werden.

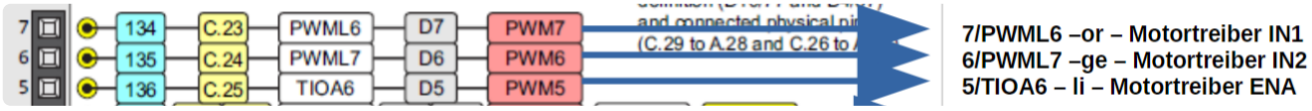
Lab 4

> In this lab you're supposed to implement the motor control.

Task 1

Schreiben/Ergänzen Sie, zur Initialisierung eines PWM-Signal (ENA) an TIOA6/PIOC25 und der für die Richtung notwendigen Signale (IN1, IN2) an PIOC23 und PIOC24, den benötigten Code in drive_init() und drive_loop().

The first step is to initialize the PWM-Signal, also referred to as the enable signal (ENA), at the TIOA6 (Pin 25 of PIO controller C) where the stepper driver is connected. The IN1 and IN2 (PIOC23 and PIOC24 respectively) refers to the signals which control the direction and speed of the motor.



ENA

The enable pin, as the name would suggest, enables the stepper motor.

IN1 & IN2

Both timing and order of these signals determines the motor's behaviour:

- Rotational direction is determined by the sequence of the signals sent
- Motor spin speed is determined by signal speed
- Rotation is determined by total amount of signals sent

Code

Initialisation

1. First while we setup the PWM-Signal we disable the pin **PIOC25** (motor). We also enable the pins **PIOC23** and **PIOC24** for the PIO controller.
   `PIOC_PER |= PIOC_PER_P25 | PIOC_PER_P23 | PIOC_PER_P24;`
2. Then we have to select the function (in this case B) with the AB select register (**ABSR**).
   `PIOC_ABSR |= PIOC_ABSR_P25;`
3. We also want to set the data direction to output and clear the data with **CODR**.
   `PIOC_OER |= PIOC_OER_P23 | PIOC_OER_P24 | PIOC_OER_P25;`
   `PIOC_CODR |= PIOC_CODR_P23 | PIOC_CODR_P24 | PIOC_CODR_P25;`
4. Then we enable the clock for the PIO C controller and the TIOA6.
   `PMC_PCER0 |= PIOC_ID;` PMC_PCER1 |= PMC_PCER1_PID33 | PMC_PCER1_PID36;`
5. Now we want to configure the PWM-Signal. First we disable the clock.
   `TC2_CCR0 = TC2_CCR0_CLKDIS;`
6. The PWM configuration is pretty much the same as with the trigger signal for the sonic sensor. Simply change the channel from 1 to 0 and from TIOB to TIOA.
7. For RC and RA:
   1. RC defines the PWM period. We calculate it as `MCK2 / MOTOR_MAX_RATE`, where `MCK2` is 42 MHz and `MOTOR_MAX_RATE` is 100. So RC's counter top value is 420.000, which corresponds to a 100 Hz PWM frequency.

    2. RA is initially set to half of RC for a 50% starting duty cycle. At runtime, RA will be adjusted by the duty cycle calculation to vary motor speed.
8. At the end we want to enable the clock again and turn on the software trigger.
   `TC2_CCR0 = TC2_CCR0_CLKEN | TC2_CCR0_SWTRG;`
   Duty cycle calculation
   We use a helper function to convert a motion value (the desired motor intensity) into an RA register value to adjust the PWM.
9. First we turn negative values positive since if we need to drive backwards we still need a positive motion value, the direction (e.g. backward) is handled later by other helper functions.
10. The value is clamped to `MOTOR_MAX_RATE` (100.0f) to prevent going out of range.
11. The duty cycle is computed as a linear interpolation between `MOTOR_MIN_RATE` and `MOTOR_MAX_RATE`. (We used 20% for our minimum duty cycle because the stepper motor in the car needs enough power to start moving, otherwise it will stall.)
12. The percentage is then converted to an absolute register value by scaling it relative to `TC2_RC0` (PWM period). This gives us the RA value to load into the timer.
    Driving behaviour
13. We used 3 functions to select the drive behaviour.

State	ENA	IN1(23)	IN2(24)	Motor behaviour
Forward	PWM (>0%)	LOW	HIGH	Rotate forward
Backward	PWM (>0%)	HIGH	LOW	Rotate reverse
Stop	PWM (0%)	LOW	LOW	Come to a stop

Drive Loop

1. First we check if **sonic** and **motion** have been enabled.
2. If they have we switch **PIOC25** to peripheral control, enabling the PWM output on **TIOA6**.
3. Then, we select our drive behaviour based on `value.motion`:
   - **Positive motion**: Call `drive_forward()` and update `TC2_RA0` with the calculated duty cycle.
   - **Negative motion**: Call `drive_backward()` and similarly update `TC2_RA0`.
   - **Zero motion**: Call `drive_stop()` and set `TC2_RA0` to 0, so duty cycle is 0%.
4. If **sonic** or **motion** is inactive, we switch **PIOC25** to PIO control, disabling the PWM output.

Task 2

Implementieren Sie die Motorsteuerung in der Funktion drive_loop. Denken Sie daran, dass Ihr Regelalgorithmus (im nächsten Termin zu implementieren) zur Ansteuerung des Motors verwendet werden soll. Die Implementierung erfolgt dann in den Dateien motion.h und motion.c.

In this task you implement the motor control logic in `drive_loop()`. The function does not decide *how fast* the motor should run on its own — it reads `value.motion`, which is later provided by the motion module (`motion_loop()` in `motion.c`). `drive_loop()` is responsible for translating that command into hardware signals: PWM duty cycle on ENA (TIOA6/PIOC25) and direction on IN1/IN2 (PIOC23/PIOC24).

Architecture

The control loop is split across two modules:

1. **`motion_loop()`** (`motion.c`): Computes the desired motor command from the ultrasonic distance measurement and the selected goal distance (`value.goal`). The result is written to `value.motion`.

2. `drive_loop()` (`drive.c`): Reads `value.motion` and drives the motor driver accordingly.

In `main.c` , both loops run every iteration — first `motion_loop()` , then `drive_loop()` — so the motor always reacts to the latest control command.

Signal	Meaning
`value.motion > 0`	Drive forward; magnitude controls PWM duty cycle
`value.motion < 0`	Drive backward; magnitude controls PWM duty cycle
`value.motion == 0`	Stop the motor and set PWM duty cycle to 0%

Code

Drive loop

1. First we check whether **sonic** and **motion** have been enabled via the serial menu (`state.sonic` and `state.motion`).
2. If both are active, we hand control of **PIOC25** to the timer peripheral so the PWM output appears on **TIOA6**: `PIOC_PDR = PIOC_PDR_P25;`
3. Based on the sign of `value.motion` , we select the drive behaviour:
   - **Positive** (`> 0.0`): call `drive_forward()` — IN1 low, IN2 high
   - **Negative** (`< 0.0`): call `drive_backward()` — IN1 high, IN2 low
   - **Zero** (`== 0.0`): call `drive_stop()` — IN1 and IN2 low, and set `TC2_RA0 = 0` for 0% duty cycle
4. For forward and backward motion, the PWM duty cycle is computed by the helper function `calculate_duty_cycle()` :
   1. Negative values are made positive (direction is handled separately by `drive_forward()` / `drive_backward()`).
   2. The value is clamped to `MOTOR_RATE` (100).
   3. The duty cycle is computed as a linear interpolation between `MOTOR_RATE_MIN` (20%) and `MOTOR_RATE` (100%): `duty_percent = MOTOR_RATE_MIN + ((MOTOR_RATE - MOTOR_RATE_MIN) * motion_value / MOTOR_RATE)` .
   4. The percentage is converted to a register value: `TC2_RA0 = TC2_RC0 * (duty_percent / MOTOR_RATE)` .
5. If **sonic** or **motion** is inactive, we switch **PIOC25** back to PIO control, disabling the PWM output: `PIOC_PER = PIOC_PER_P25;`

Motion module interface

The regulation algorithm is implemented in `motion_loop()` (`motion.c`). It runs only when a new sonic measurement is available (`value.sonic.new`) and motion control is enabled.

1. The controlled variable is the measured distance `value.sonic.distance` (in mm).
2. The setpoint is the selected goal distance `value.goal` (set via keys `1 – 9` in the serial menu).
3. The control error is computed as `error = setpoint - distance` .
4. If the error is within the **deadzone** (±10 mm), the motor is stopped (`value.motion = 0`) and the integral term is reset to avoid jitter around the setpoint.
5. Otherwise a PID controller calculates the manipulated variable:
   - `p = error * KP` (with `KP = 0.1`)
   - `i = KI * integral` (with `KI = 0` , so this term has no effect)
   - `d = KD * (error - error_last) / dt` (with `KD = 0` , so this term has no effect either)
6. Near the setpoint, the output is scaled down within the **slowdown range** (150 mm): `scale = abs_error / SLOWDOWN_RANGE` , then `pid *= scale` . This reduces motor speed as the car approaches the target distance.
7. The result is clamped to `±MOTOR_MAX_RATE` (100) and stored in `value.motion` , which `drive_loop()` consumes on the next iteration.

PID controller

The motion module uses a P(I)D controller — a continuous controller where the control variable can take any value within the control range (greater precision than a simple on/off switch).

- **Controller**: Device or algorithm that adjusts a physical quantity (actual value) to match a target value (setpoint).
- **Controlled System**: A real system influenced by external disturbances, controlled by a controller.
- **Feedback**: A control loop concept where a system's output is fed back to regulate its behaviour.
- **Controlled variable y(t)**: The physical quantity (actual value) being regulated — here: the measured distance.
- **Setpoint w(t)**: The desired target value for the controlled variable — here: `value.goal`.
- **Control error e(t) = w(t) – y(t)**: The difference between the setpoint and the actual value.
- **Manipulated variable u(t)**: The controller's output (adjustment signal) — here: `value.motion`.
- **Disturbance d(t)**: External influence affecting the controlled system.

P - proportional

This represents the part where the manipulated variable u(t) is influenced by the control error e(t), which is the difference between the setpoint (desired value) and actual value.

I - integral

This represents the part where the accumulated control error over time is used to eliminate a persistent offset between the setpoint and the actual value. In our implementation `KI` is set to 0, so this term has no effect.

D - derivative

This represents the part where the rate of the control error e(t) is used to calculate by how much the manipulated variable u(t) should be adjusted to dampen oscillations and counteract rapid changes in the error, thereby reducing overshoot when approaching the setpoint. In our implementation `KD` is set to 0, so this term has no effect either.

Leitfragen

- Wie könnte sich das Ansprechverhalten des Motors zwischen dem Fahrzeug und dem festen Versuchsaufbau unterscheiden?
  - The car and the lab setup are different because the car adds a certain weight onto the motor, increasing the friction resistance for the tires. Thus the controller needs different input parameters, e.g. increase the amount of spinning in the motor, to ensure the car starts moving.
- Welche anderer Möglichkeiten gäbe es, den Treiberbaustein des Motors anzusprechen?
  - You could use *Step/Dir* where you have a step pin which receives a signal and a dir pin that is either low or high. The step signal controls the speed through the amount of signal pulses it sends to the pin with one pulse corresponding to one step of the motor. The dir pin is responsible for the direction, so e.g. low would mean forward and high reverse.
- Für welche Frequenz für das PWM-Signal haben Sie sich entschieden?
  - Our PWM signal has a frequency of 100Hz (MCK2 / 100).